1. Project Overview

MicroTask is a student-focused micro-task marketplace platform where tasks are **AI-classified, auto-priced, and auto-matched** — no bidding, no negotiation. It serves two distinct user roles: **Students** (who complete tasks and earn money) and **Clients** (external businesses/individuals who post tasks).

2. Technology Stack

Layer	Technology	Status
Frontend	Next.js 16.1.6 (App Router), React 19, TypeScript	✅ Built
Styling	Tailwind CSS 4, PostCSS	✅ Built
Backend	Node.js, Express.js	✅ Built
Database	MongoDB (Mongoose 8.2.0)	✅ Built
AI Classification	OpenAI GPT-3.5-turbo + n8n Webhook	✅ Built
Job Queue	BullMQ + Redis (ioredis)	✅ Built
Auth	JWT + bcryptjs	✅ Built
Deployment	Vercel-ready (serverless export)	✅ Configured

3. Backend — What's Done

3.1 Data Models (4 models)

Model	Fields	Status
User	name, email, password_hash, college_domain, company_name, phone, role (student/client), isVerified, depositPaid, skill_tags, reliability_score, skill_tier, weekly_hours_completed, wallet_balance (default ₹500), badges, created_at	✅ Complete
Client	name, email, password_hash, company_name, phone, role (client), wallet_balance (default ₹1000), total_tasks_posted, total_spent, is_verified, created_at	✅ Complete
Task	title, description, category, complexity_level (Low/Medium/High), estimated_time_hours, price, ai_suggested_price, final_price, posted_by, assigned_to, status (open/assigned/completed/cancelled), digital_or_physical, created_at	✅ Complete

Transaction	task_id, payer_id, receiver_id, amount, status (escrow/ released/refunded), created_at	✅ Complete

3.2 API Routes (6 route files, ~15 endpoints)
Student Auth (/api/auth)

Endpoint	Method	What It Does	Status
/api/auth/register	POST	Register with college-verified email only	✅
/api/auth/login	POST	Login, returns JWT (7-day expiry)	✅

Client Auth (/api/client/auth)

Endpoint	Method	What It Does	Status
/api/client/auth/ register	POST	Register with ANY email (no domain restriction)	✅
/api/client/auth/login	POST	Login, returns JWT + depositPaid status	✅

Tasks (/api/tasks)

Endpoint	Method	What It Does	Status
/api/tasks	POST	Create task → AI classifies → backend prices → auto-match → escrow deduct	✅
/api/tasks	GET	List all open tasks	✅
/api/tasks/my	GET	Tasks posted by or assigned to current user	✅
/api/tasks/:id/ complete	POST	Mark complete → release escrow → update reliability/tier → award badges	✅
/api/tasks/:id/cancel	POST	Cancel → refund poster → penalize assignee reliability	✅

Users (/api/users)

Endpoint	Method	What It Does	Status
/api/users/me	GET	Current user profile (excluding password)	✅
/api/users/wallet	GET	Wallet balance + full transaction history	✅

Client Users (/api/client/users)

Endpoint	Method	What It Does	Status
/api/client/users/ deposit	POST	Mark client deposit as paid	✅

/api/client/users/me	GET	Client profile	✅
/api/client/users/stats	GET	Full dashboard stats (total posted, active, completed, cancelled, total spent, recent tasks)	✅
/api/client/users/wallet	GET	Client wallet + transactions	✅

Admin (/api)

Endpoint	Method	What It Does	Status
/api/reset-week	POST	Reset weekly_hours_completed for all users to 0	✅

3.3 Business Services (4 services)

Service	What It Does	Status
aiService.js	Dual-mode AI classification: (1) Keyword overrides for common tasks (chai, coding, design, etc.), (2) n8n webhook call for complex tasks, (3) OpenAI GPT-3.5-turbo fallback for BullMQ worker. Graceful degradation if AI is unavailable.	✅ Complete
pricingService.js	Category-based pricing with base rates (delivery ₹50/hr, coding ₹300/hr, design ₹250/hr, etc.) × complexity multiplier (Low 1.0, Medium 1.3, High 1.6). Clamped between ₹20–₹2000.	✅ Complete
matchingService.js	Auto-assigns tasks to highest-reliability, highest-tier student who hasn't exceeded 6hr/week cap.	✅ Complete
badgeService.js	Awards badges: "Consistent Contributor" (≥5 tasks), "On-Time Pro" (reliability ≥10), "Academic Safe Worker" (stayed within weekly cap). Calculates tier progression (Tier 1→2→3).	✅ Complete

3.4 Job Queue System

Component	What It Does	Status
taskQueue.js	BullMQ queue with 3 retry attempts, exponential backoff (2s base), auto-remove on complete	✅ Complete
redis.js	Redis connection manager via ioredis	✅ Complete

taskWorker.js	Background worker: re-classifies tasks via OpenAI, recalculates price, re-matches to worker, updates DB. Runs with concurrency of 5.	✅ Complete

3.5 Middleware (3 files)

Middleware	What It Does	Status
auth.js	JWT verification for student routes	✅
clientAuth.js	JWT verification for client routes (checks role=client)	✅
domainCheck.js	Restricts student registration to college.edu and university .ac.in domains	✅

4. Frontend — What's Done

4.1 Shared Components

Component	What It Does	Status
Sidebar.tsx	Left navigation sidebar with links to Dashboard, Marketplace, Profile, Wallet	✅
Footer.tsx	Global footer component	✅
LogoutButton.tsx	Logout handler (clears localStorage)	✅

4.2 API Client (lib/api.ts)

Centralized apiFetch wrapper with JWT auto-injection. Covers all 11 API endpoints for both student and client roles. Includes saveToken, saveClientToken, logout, and getUserRole helpers. ✅ **Complete**

4.3 Student Pages (8 pages)

Page	Route	What It Does	Dynamic Data?	Status
Landing	/	Hero section, features bento grid (AI Classification, Secure Escrow, Trusted Profiles), "How It Works" steps, testimonials, CTA	Static (showcase)	✅ Built
Login	/login	Email + password form, calls /api/auth/login, saves JWT, redirects to /dashboard. Links to client portal.	✅ Dynamic	✅ Built

Register	/register	Name, college email, password, skill tags. Calls /api/auth/register, saves JWT, redirects to /dashboard.	✅ Dynamic	✅ Built
Dashboard	/dashboard	Sidebar layout, hero welcome, create-task quick input, AI suggestion panel, active task cards with progress rings, earnings summary with mini chart, skills widget, recent activity logs	Static (hardcoded demo data)	⚠️ Partially Dynamic
Post Task	/tasks/post	Title, description, digital/physical selector. Calls /api/tasks → shows AI classification result (category, complexity, hours, auto-price, assigned worker). Client-only gated.	✅ Dynamic	✅ Built
Profile	/profile	Sidebar layout, profile hero with avatar/rating/earnings, earnings performance chart (SVG), stats cards (completed tasks, success rate, avg completion), pie chart by category, task comparison bar chart, recent task history	Static (hardcoded demo data)	⚠️ Partially Dynamic

Wallet	/wallet	Sidebar layout, total balance card with VISA badge, payout schedule with progress bar, earnings growth SVG line chart, transaction activity log with status icons	Static (hardcoded demo data)	⚠️ Partially Dynamic
Marketplace	/marketplace	Sidebar layout, search bar, task grid cards (Development $120, Creative $45, Delivery $15, Writing $80), "Become a Verified Expert" promotional banner	Static (hardcoded demo data)	⚠️ Partially Dynamic

4.4 Client Pages (7 pages)

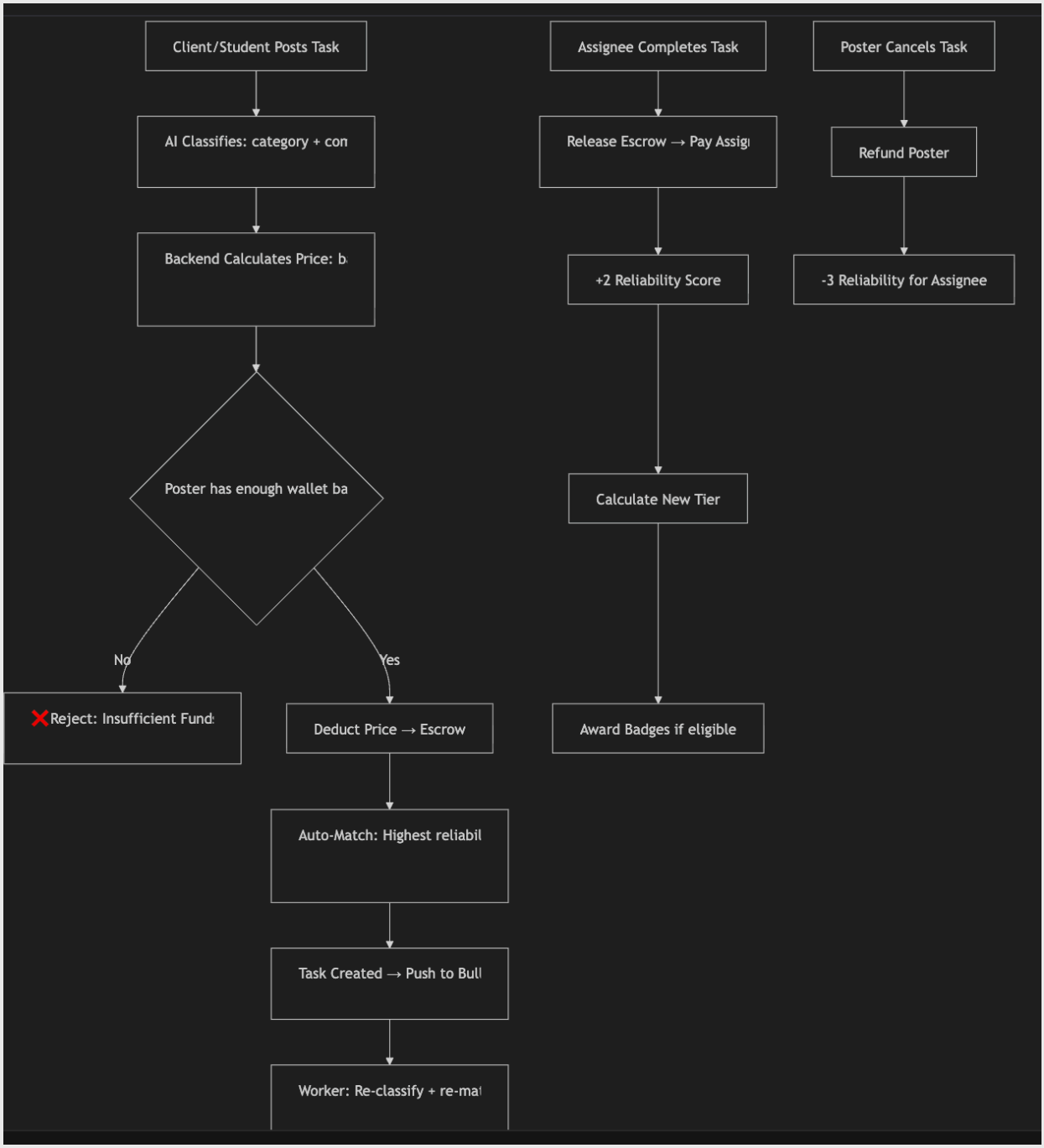
Page	Route	What It Does	Dynamic Data?	Status
Client Login	/client/login	Email + password → /api/client/auth/login → saves client JWT	✅ Dynamic	✅ Built
Client Register	/client/register	Name, email, password, company, phone → /api/client/auth/register	✅ Dynamic	✅ Built
Client Deposit	/client/deposit	Pay deposit gate → /api/client/users/deposit → redirects to dashboard	✅ Dynamic	✅ Built
Client Dashboard	/client/dashboard	Fetches live stats (wallet balance, total posted, active, completed, total spent). Tabbed recent tasks list (all/active/completed) with status badges. Quick-action cards.	✅ Fully Dynamic	✅ Built
Client Profile	/client/profile	Client profile view	✅ Dynamic	✅ Built

Client Wallet	/client/wallet	Client wallet + transactions	✅ Dynamic	✅ Built
Client Layout	/client/ layout.tsx	Shared layout wrapper + ClientNavBar	—	✅ Built

4.5 Additional Pages

Page	Route	Status
Create Task	/create	✅ Built
Dashboard Analytics	/dashboard/analytics	✅ Built
Available Tasks	/tasks/available	✅ Built
My Tasks	/tasks/my	✅ Built

5. Core Business Logic — Fully Implemented




```
Client/Student Posts Task
AI Classifies: category + complexity + hours
Backend Calculates Price: base_rate × hours × complexity_multiplier
```

```
Poster has enough wallet balance?
❌ Reject: Insufficient Funds
Deduct Price → Escrow
Auto-Match: Highest reliability student under 6hr/week cap
Task Created → Push to BullMQ Queue
Worker: Re-classify + re-match in background
Assignee Completes Task
Release Escrow → Pay Assignee
+2 Reliability Score
Calculate New Tier
Award Badges if eligible
Poster Cancels Task
Refund Poster
-3 Reliability for Assignee
```

Pricing Table (Backend-Controlled)

Category	Base Rate (₹/hr)	Low (×1.0)	Medium (×1.3)	High (×1.6)
Delivery	₹50	₹50/hr	₹65/hr	₹80/hr
Coding	₹300	₹300/hr	₹390/hr	₹480/hr
Design	₹250	₹250/hr	₹325/hr	₹400/hr
Writing	₹200	₹200/hr	₹260/hr	₹320/hr
Research	₹180	₹180/hr	₹234/hr	₹288/hr
Teaching	₹220	₹220/hr	₹286/hr	₹352/hr
General	₹150	₹150/hr	₹195/hr	₹240/hr

All prices clamped to **₹20 minimum, ₹2000 maximum**.

6. What's Working End-to-End ✅

1. **Student Registration** with college domain verification
2. **Client Registration** with any email (no restriction)
3. **JWT Authentication** for both student and client flows
4. **Task Creation** with full AI pipeline (keyword override → n8n webhook → OpenAI fallback)
5. **Automatic Pricing** by the backend (AI has no say in price)
6. **Auto Worker Matching** based on reliability score + tier + weekly cap
7. **Escrow System** — deduct on creation, release on completion, refund on cancellation
8. **Task Completion** — escrow release, reliability bump, tier recalculation, badge awarding
9. **Task Cancellation** — poster refund, assignee reliability penalty
10. **Client Dashboard** — fully dynamic stats from the API
11. **Client Deposit Gating** — clients must pay deposit before posting
12. **BullMQ Background Processing** — async re-classification with 5 concurrent workers
13. **Vercel-Ready Deployment** — conditional server listen for serverless vs local

7. What Needs Work ⚠️

Area	Issue	Severity

Student Dashboard	Displays hardcoded mock data instead of calling api.getMe(), api.getMyTasks(), api.getWallet()	🟡 Medium
Student Profile	Shows hardcoded "Alex Sterling" with static stats. Should fetch real user data.	🟡 Medium
Student Wallet	Shows hardcoded "$4,822.50" balance and fake transactions. Should call api.getWallet().	🟡 Medium
Marketplace	Shows hardcoded task cards. Should call api.getOpenTasks() and render real tasks.	🟡 Medium
Domain Whitelist	Only allows college.edu and university.ac.in. Needs to be expanded or made configurable.	🟢 Low
.env.example	Missing REDIS_URL and FRONTEND_URL entries	🟢 Low
Error Handler	Leaks err.stack to console in production — should be suppressed	🟢 Low
npm Vulnerabilities	Backend: 4 (3 high), Frontend: 6 (3 high) — fixable with npm audit fix	🟡 Medium
No Tests	Zero unit or integration tests exist	🟡 Medium
No Rate Limiting	No rate limiting on any API endpoint	🟡 Medium

8. File Inventory
Backend (31 files across 7 directories)

```
backend/
├── .env.example
├── .gitignore
├── package.json
├── vercel.json
├── test-queue.js
└── src/
    ├── index.js                    # Express entry + MongoDB connect +
Vercel export
    ├── middleware/
    │   ├── auth.js                 # Student JWT middleware
```

```
    │   ├── clientAuth.js          # Client JWT middleware
    │   └── domainCheck.js         # College email whitelist
    ├── models/
    │   ├── User.js                # Student/Client unified model
    │   ├── Client.js              # Client-specific model (unused
duplicate?)
    │   ├── Task.js                # Task model with AI fields
    │   └── Transaction.js         # Escrow transaction model
    ├── routes/
    │   ├── auth.js                # Student register/login
    │   ├── clientAuth.js          # Client register/login
    │   ├── clientUsers.js         # Client profile/stats/wallet/deposit
    │   ├── tasks.js               # Full task CRUD + lifecycle
    │   ├── users.js               # Student profile/wallet
    │   └── admin.js               # Weekly reset
    ├── services/
    │   ├── aiService.js           # n8n + OpenAI + keyword overrides
    │   ├── pricingService.js      # Category × complexity pricing
    │   ├── matchingService.js     # Reliability-based auto-match
    │   └── badgeService.js        # Achievement badges + tier calc
    ├── queues/
    │   ├── redis.js               # Redis connection
    │   └── taskQueue.js           # BullMQ queue config
    └── workers/
        └── taskWorker.js          # Background task processor
```

Frontend (26+ pages/components)

```
frontend/
├── app/
│   ├── page.tsx                   # Landing page
│   ├── layout.tsx                 # Root layout
│   ├── globals.css                # Global styles
│   ├── login/page.tsx             # Student login
│   ├── register/page.tsx          # Student registration
│   ├── dashboard/
│   │   ├── page.tsx               # Student dashboard
│   │   └── analytics/page.tsx     # Analytics view
│   ├── profile/page.tsx           # Student profile
│   ├── wallet/page.tsx            # Student wallet
│   ├── marketplace/page.tsx       # Task marketplace
```

```
│   ├── create/page.tsx            # Create task
│   ├── tasks/
│   │   ├── post/page.tsx          # Post task (AI flow)
│   │   ├── available/page.tsx     # Browse open tasks
│   │   └── my/page.tsx            # My tasks
│   ├── client/
│   │   ├── layout.tsx             # Client layout wrapper
│   │   ├── ClientNavBar.tsx       # Client navigation
│   │   ├── login/page.tsx         # Client login
│   │   ├── register/page.tsx      # Client registration
│   │   ├── deposit/page.tsx       # Deposit gate
│   │   ├── dashboard/page.tsx     # Client dashboard (fully dynamic)
│   │   ├── profile/page.tsx       # Client profile
│   │   └── wallet/page.tsx        # Client wallet
│   └── components/
│       ├── Sidebar.tsx            # Navigation sidebar
│       ├── Footer.tsx             # Footer component
│       └── LogoutButton.tsx       # Logout handler
└── lib/
    └── api.ts                     # Centralized API client (11 endpoints)
```

9. Summary

Metric	Count
Backend API Endpoints	~15
Database Models	4
Business Services	4
Frontend Pages	18+
Reusable Components	3
Background Workers	1 (5 concurrency)
Total Source Files	~50+

Overall Completion: **~75%**

The **backend is essentially production-ready** — all business logic (AI classification, pricing, matching, escrow, badges, tiering, job queues) is fully implemented and end-to-end functional.

The **client-side frontend is fully dynamic** — login, register, deposit, dashboard, wallet, and profile all fetch real API data.

The **student-side frontend** has all pages built with premium UI designs, but **4 key pages** (Dashboard, Profile, Wallet, Marketplace) still display **hardcoded demo content** instead of fetching live data from the backend APIs. Wiring these pages to the existing `api.ts` methods is the primary remaining work.